Deliverable 2

Adrian N. Vangen and Viktor M.L. Hedman

Imports

In [93]:
```
import json
import pandas as pd
import numpy as np
from pathlib import Path
import re
```

Task 1,1: File reader function

In [94]:
```
def filereader(file:str) -> tuple:
    """
    This function take a string contaning the path of the file, and check the en
    The function checks the encoding used by using a loop tha try the diffrent e
    stors it in a list to geder wit the contents od the file.
    The function then returns that list contaning the contents of the file and t
    The function also assume the only to encodings are utf-8 and latin-1.
    """
    path=Path(file) #Cheeks the path

    encodings = ('utf-8', 'latin-1') #List of the encodings

    #A loop that lops through the encodings
    for encoding in encodings:
        #Trys to open and read the encoding
        try:
            with open(path, 'r', encoding = encoding) as file:
                file_text_ontents=file.read()#Reed the fil as one long string

        #Continues if the encoding does not work
        except UnicodeDecodeError:
            continue

    #Returns a tupel containing the file string and encoding type
    return (file_text_ontents, encoding)
```

Task1,2: Log parser function

In [95]:
```
file_text=filereader('Data\\2026\\library_cafe\\POS_01\\register_sales.log')[0]

def log_parser(file_text: str) -> list[dict]:
    """
    The function take a the contents of a sale file and returns
    a list containing dictionarys with valid sales.
    First it split the lines and make an enpty record list.
    Then it define the reguler expressions.
    Lastly the function uses a loop to cheek and
    store the matching lines as a list of dictionerys.
    """
```

```
    file_lines=file_text.splitlines() # Slit the text into lines

    recordslist=[] #Define the list for the record logs.

    #This list of regulare expressions are made using gemini ai
    log_pattern = re.compile(
    r'^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s+'
    r'\[(?P<terminal>[A-Za-z0-9_]+)\]\s+'
    r'item=(?P<item>.+?)\s+'
    r'price=(?P<price>\d+(?:\.\d+)?)\s+'
    r'(?P<currency>[A-Z]{3})\s+'
    r'\((?P<status>[A-Z]+)\)$'
    )

    for line in file_lines:

        #Cheek if the line match eny of the regular expressions and stores the m
        match=log_pattern.match(line)

        #If it match it makes in to a dictionery and appends it in to the record
        if match:
            match_dict=match.groupdict()
            recordslist.append(match_dict)

    return recordslist #Returns the record list
```

Task 2

In [96]:

```
def recursive_directory_search(input_path: str) -> list[Path]:
    """
    This function takes in an input_path (text string) which is the main folder.
    recursively through all nested subfolders inside the main folder. Returns a
    to the files that has .log at the end of its name.
    """
    #Recursive: you call the function inside the function
    path_object = Path(input_path) #Making the main directory into a path object
    found_files = [] #During the searching, this empty list will collect and sto

    for element in path_object.iterdir(): #Iterates through every single item in
        if element.is_dir(): #Check if the item is a directory (subfolder)
            found_files.extend(recursive_directory_search(element)) #Recursively
            #extend() merges multiple items into a list.

        #Check if the item is a file and if the filename ends with .log
        elif element.is_file() and element.name.endswith(".log"):
            found_files.append(element) #Add the file's Path object to the founa

    return found_files #Returns the list of all the log files


def recursive(input_path: str | Path | dict | list) -> set[str]:
    """
    The input for this function can be a file path string, a pathlib.Path object
    a Python dictionary or a Python list. This recursive function is able to nav
    the data structure of unknown depth in the store_config.json file.
    This means that the functions can walk through any combination and depth of
    Returns a set of all register id strings where "active" is true.
    """
```

```
    if isinstance(input_path, (str, Path)): #Check if the input is a text string
        path_object = Path(input_path) #Convert the input into a pathlib.Path ob
        if path_object.is_file(): #Check if the path object gives us a real file
            with open(path_object, "r", encoding="utf-8") as file: #Read the jso
                json_data = json.load(file)
            return recursive(json_data) #The loaded json_data is sendt to this r

    result = set() #Sets only store unique values

    if isinstance(input_path, dict): #Check if the data is a dictionary
        if input_path.get("active") is True: #Check if this dictionary contains
            if "id" in input_path and isinstance(input_path["id"], str):
                result.add(input_path["id"])
        for key, value in input_path.items(): #Loop through every key-value pair
            result.update(recursive(value)) #Merge founded IDs into the result s

    elif isinstance(input_path, list): #If the data is a list
        for item in input_path: #Loop through every item in the list
            result.update(recursive(item))

    return result
```

Task 3: Functions for cleaning and analysing data using NumPy

In [97]:

```
def remove_outliers(df: pd.DataFrame) -> np.ndarray:
    """
    Takes a dataframe as an input. This function convert input price data into
    1D NumPy array. Price data that are more than 3 standard deviations away fro
    are called outliers. This function replaces all the outliers with the median
    Returns a cleaned 1D array where all outliers are removed and replaced with
    """
    price_array = np.array(df['price']) #The 'price' column in the dataframe is

    my = np.average(price_array) #my means average
    sigma = np.std(price_array) #sigma means standard deviation

    outliers = abs(price_array-my) > 3 * sigma #numpy does this element-wise so
    #outliers is a 1D numpy array contaning bolean True or False values.

    median = np.median(price_array)

    price_array[outliers] = median #Here we replace the outlier values with the

    cleaned_array = price_array
    return cleaned_array



def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> n
    """
    This funciton takes the numpy arrays sales_count and refund_count as input.
    Returns the computed element-wise refund_ratio.
    """
    refund_ratio = refund_count/(sales_count + 10**(-6)) #return_ratio is numpy
    return refund_ratio
```

Task 4: Functions that uses pandas

In [98]:
```python
def Dataframe_conv(li: list[dict[str, str]]) -> pd.DataFrame:
    """Convert a list of dictionaries into a pandas DataFrame.
    It uses the function DataFrame and uses the function to_date time
    to convert the timestamp values to the date time format.
    The function also sorts the dataframe by the time in ascending order.
    """

    df=pd.DataFrame(li) # Makes the dataframe using the list
    df['timestamp'] = pd.to_datetime(df['timestamp']) # Convert the timstamps to
    df['price'] = pd.to_numeric(df['price']) # Convert the price to numbers
    df_shorted = df.sort_values(by='timestamp', ascending = True) # Sorts the da

    return df_shorted

def resampeld_report(df: pd.DataFrame) -> pd.DataFrame:
    """
    The Function grops the data by item, resampel the transactions
    into 1 hour windows and shows the total price.
    First the function converts the price to numbers
    and set the index to the timstamps.
    Then it grups the data baist on items, sets the data in 1_hour windows
    and calculates the total price. Lastly the grupped data frame is being retur
    """

    df = df.set_index('timestamp') #set the index to the timestamps

    #Grups the data and take the valus for every houre
    df = df.groupby(['item', pd.Grouper('timestamp', freq='1h')])['price'].sum()

    return df #Returns the new dataframe

def pivot_table(df: pd.DataFrame) -> pd.DataFrame:
    """
    This function uses the pivot_table function bilt in to pandas to make a pivo
    The pivot table is maid using the pivot_table function includet in the panda
    """

    #Creates the pivot table
    pivot_tab = df.pivot_table(values = 'price', columns = 'item', index = 'stat

    return pivot_tab #Returns the pivot table

def refund_ratio_df(df: pd.DataFrame) -> pd.DataFrame:
    """
    This function uses the refund ratio function vi definf to calculate
    the refundratio to the of the prices per item in the data frame.
    """

    #Make som enty lists for the ratios and items
    rl=[]
    il=[]

    #The loop take the unique valus of the items in the data frame.
    for item in list(df['item'].unique()):
        #Calculate the counts of each status for the item
        count = df[df['item'] == item]['status'].value_counts()
        #Calculates the refund ratio using the function defined erlyer
        ratio = compute_refund_ratio(count['SUCCESS'], count['REFUND'])
        #Appends the ratio and item into the aproptiat list
```

```
        rl.append(ratio)
        il.append(item)

    ratio_dict={'item':il, 'ratio':rl} #Makes the list into a dictionary


    #Returns the dataframe with the refund ratios addet and a separar rafund rat
    return pd.DataFrame(ratio_dict)
```

Task 5

In [99]:

```
#Calls the recursive_directory_search to recursivly put all the paths in a list
path_list = recursive_directory_search("Data")
#The filepath for the store_config.json file
file_path = Path("Data") / "store_config.json"
#Calls the recursive to recursivly find all the ID that are active
Register_id = recursive(file_path)

dict_list=[] #define a dictionery list to use for making the data frame

#A Loop goes throug all the paths and append the dictionerys form the log functi
for path_str in path_list:
    file_text=filereader(path_str)[0]
    log=log_parser(file_text)
    dict_list.append(log)

#define a dataframe for later use
df=pd.DataFrame()

#A loop that uses the Datafreme converter funktion to make a dataframe of all th
for d in dict_list:
    df=pd.concat([df, Dataframe_conv(d)])

#Shorts the dataframe and cleans it using the remove outliers funktion
df=df.sort_values(by='timestamp', ascending = True).reset_index()
df['price'] = remove_outliers(df)

#Makes the value of the price of all the NOK currensys to be 10 times bigger
df.loc[df['currency'] == 'NOK', 'price'] = df.loc[df['currency'] == 'NOK', 'pric

#Uses the resampeld_report to make a raport dataframe
raport = resampeld_report(df)
#Uses the poviot table function to make a poviot table
tabel=pivot_table(df)
#Uses the refund ratio function to make a refund data frame
ratios = refund_ratio_df(df)

#Checks if evryting gets done corectly
print(df.head(5))
print(raport.head(5))
print(tabel)
print(ratios.head(5))

#Write all the output files using the apropriate dataframes
df.to_csv('cleaned_transactions.csv')
raport.to_json('hourly_revenue.json', indent = 2)
tabel.to_csv('status_pivot_table.csv')
ratios.to_csv('item_refund_ratios.csv')
```

```
   index           timestamp terminal        item  price currency     status
0      0 2026-10-15 07:00:11   POS_01      Muffin   3.88      EUR    SUCCESS
1      0 2026-10-15 07:00:14   POS_03  Cappuccino   4.40      USD    SUCCESS
2      0 2026-10-15 07:00:45   POS_01    Espresso   2.77      USD    SUCCESS
3      0 2026-10-15 07:01:09   POS_02    Espresso   3.15      USD    SUCCESS
4      1 2026-10-15 07:01:11   POS_01      Muffin  38.60      NOK  CANCELLED
         item           timestamp   price
0  Cappuccino 2026-10-15 07:00:00  807.77
1  Cappuccino 2026-10-15 08:00:00  885.52
2  Cappuccino 2026-10-15 09:00:00  255.34
3   Croissant 2026-10-15 07:00:00  731.10
4   Croissant 2026-10-15 08:00:00  652.72
item       Cappuccino  Croissant  Espresso    Latte   Muffin  Sandwich
status
CANCELLED      236.69     268.83    185.10   462.81   220.66    299.20
REFUND         603.16     135.78    221.96   243.58   457.08   1238.49
SUCCESS       1108.78    1041.94    842.72  1655.03  1218.12   2375.44
         item     ratio
0      Muffin  0.376623
1  Cappuccino  0.578125
2    Espresso  0.225000
3       Latte  0.193548
4    Sandwich  0.313253
C:\Users\adria\AppData\Local\Temp\ipykernel_2532\199204776.py:45: Pandas4Warning:
The default 'epoch' date format is deprecated and will be removed in a future ver
sion, please use 'iso' date format instead.
  raport.to_json('hourly_revenue.json', indent = 2)
```

We se that the that the dataframe gets properly generated and cleand. The part also gets maid corectly, and the pivot table looks right. Also se that the refund ratios looks big. It could be becaus of some calculatiom errors or that the dataframe contains all the logs form all the registers. Also se that the output files is generated nicely.